AutoML에서 Autoresearch로

DOE는 Research Agent의 Harness가 될 수 있는가?

- AutoML, MLOps, Research Workflow, DOE-guided Agent
- 실험 결과는 이후 채울 placeholder deck

시각 placeholder

- 좌: AutoML / Coding Agent
- 우: Research Loop / DOE Harness

발표 메모

- 핵심 주장은 단순하다.
- Coding Agent는 강한 harness 위에서 발전했지만, Research Agent는 아직 실험 harness가 약하다

1. 왜 지금 이 주제인가?

- Coding Agent는 실제 개발 workflow 안에서 빠르게 발전하고 있다.
- Research automation도 논문 정리, 실험 실행, 보고서 작성까지 넓어지고 있다.
- 하지만 연구 실험은 여전히 ad hoc 반복으로 흐르기 쉽다.
- 이제 질문은 "agent가 실험을 돌릴 수 있는가?"가 아니다.
- 질문은 "Research Agent를 믿게 만드는 harness는 무엇인가?"이다.

시각 placeholder

- 좌: Coding Agent stack
- 우: Research Agent stack
- 하단: "Harness gap?"

발표 메모

- 청중이 이 발표를 들어야 하는 이유를 먼저 분명히 한다.

2. AutoML이란 무엇인가?

- AutoML은 모델 개발 과정의 일부 탐색을 자동화한다.
- 대표 대상은 model selection, hyperparameter tuning, pipeline search다.
- 더 넓게는 feature engineering과 NAS까지 확장된다.
- 핵심은 비교적 주어진 search space 안에서 성능을 최적화하는 것이다.

시각 placeholder

- Data → model/pipeline search → evaluation → best configuration

발표 메모

- 이후 Autoresearch와 대비되도록 최소한의 배경만 준다.

3. 대표적인 AutoML 방법

- Grid / Random Search: 단순하고 강건하지만 비효율적일 수 있다.
- Bayesian Optimization: sample-efficient하지만 구조적 edit 공간에는 약할 수 있다.
- Evolutionary Search: 구조 탐색에 강하지만 비용이 크다.
- Multi-fidelity: 예산 효율이 좋지만 proxy bias를 안을 수 있다.
- NAS: architecture 탐색에 강하지만 탐색 비용이 높다.

시각 placeholder

- 표: 방법 / 강점 / 약점

발표 메모

- 여기서의 포인트는 방법론 나열이 아니라, 탐색 전략과 시스템 인프라가 이미 중요하다는 점이다.

4. AutoML과 MLOps의 연결

- AutoML만으로는 충분하지 않고, 실행 인프라가 함께 있어야 한다.
- experiment tracking, orchestration, reproducibility가 탐색을 실제 시스템으로 만든다.
- model registry, deployment, monitoring이 탐색을 운영으로 이어 준다.
- cost governance도 중요하다. 탐색은 수학 문제이기 전에 시스템 workload이기 때문이다.

시각 placeholder

- 상단: search logic
- 중단: tracking / orchestration
- 하단: deployment / monitoring / cost

발표 메모

- AutoML이 practical해진 이유는 알고리즘만이 아니라 harness가 붙었기 때문이다.

5. 대학원생의 연구 루프

- 문헌 조사
- 가설 수립
- 실험 설계
- 구현 및 실행
- 분석
- 토론 및 수정
- 다음 실험

시각 placeholder

- 7단계 원형 loop diagram

발표 메모

- "연구"를 하나의 optimization call이 아니라 반복 loop로 이해시키는 슬라이드다.

6. Autoresearch란 무엇인가?

- Autoresearch는 단순한 hyperparameter search를 넘는다.
- code, module, pipeline structure까지 수정할 수 있다.
- tool use, memory, literature, iterative reasoning을 함께 사용할 수 있다.
- 따라서 search space는 AutoML보다 넓고, 덜 정돈되어 있으며, 더 open-ended하다.

시각 placeholder

- hypothesis, code edit, pipeline, experiment program을 함께 포함하는 확장된 loop

발표 메모

- AutoML은 주어진 공간 안을 최적화하고, Autoresearch는 그 공간 자체를 더 넓게 다룬다.

7. Autoresearch 현황

- 이미 공개 생태계는 단일 demo를 넘어 layered ecosystem으로 분화되고 있다.
- `awesome-autoresearch`는 descendants, research-agent systems, hardware forks, domain-specific adaptations, benchmarks로 정리한다.
- `Awesome-Auto-Research-Tools`는 end-to-end systems, deep research, experiment/code agents, skills/plugins로 정리한다.
- 즉 현재의 흐름은 "하나의 agent"보다 "생태계 + 실행 인프라 + 평가 체계" 쪽으로 움직이고 있다.
- 이 발표의 DOE 논의도 그 생태계 안에서 harness 층을 다루는 제안으로 볼 수 있다.

시각 placeholder

- ecosystem map
- 상단: End-to-End Research Systems
- 좌하단: Deep Research
- 우하단: Experiment & Code Agents
- 하단: Skills / Benchmarks / Hardware Ports

8. 사용례와 발전 방향

- 사용례는 이미 다양하다: ML 실험 자동화, quant/Kaggle automation, literature synthesis, paper writing, peer review 보조.
- domain 확장도 보인다: GPU kernel optimization, voice agent hardening, genealogy, trading, solver optimization.
- 발전 방향도 분명하다: end-to-end automation, deeper experiment execution, reusable skills/plugins, stronger benchmark와 observability.
- 다시 말해 Autoresearch는 "한 번의 논문 데모"가 아니라, 점점 programmable research stack으로 이동 중이다.

시각 placeholder

- 좌: use case grid
- 우: direction arrows
- 아래 예시 태그: `RD-Agent`, `GPT Researcher`, `AIDE`, `autokernel`, `AI-Research-SKILLs`

9. AutoML vs. Autoresearch

- AutoML의 주된 탐색 대상은 config와 architecture다.
- Autoresearch의 탐색 대상은 hypothesis, code, module, pipeline, experiment program 까지 넓다.
- AutoML은 explicit objective 최적화가 중심이다.
- Autoresearch는 objective뿐 아니라 reasoning, iteration, scientific discipline까지 포함한다.
- 그래서 Autoresearch에는 search infra만이 아니라 behavioral constraint도 필요하다.

시각 placeholder

- 표: 탐색 대상 / 평가 / 인간 역할 / 위험 / 필요한 인프라

발표 메모

- 여기서부터 발표의 중심 질문은 "이 넓은 탐색을 무엇으로 제어할 것인가?"로 바뀐다.

10. Harness gap

- Coding Agent는 TDD, CI, regression test, lint, deployment check 같은 harness 위에서 발전했다.
- 신뢰를 만든 것은 모델 자체만이 아니라 검증 구조였다.
- 반면 Research Agent는 아직 experiment discipline, robustness check, reproducible iteration 구조가 표준화되어 있지 않다.
- 그래서 성능이 좋아 보여도 search가 coherent한지 판단하기 어렵다.

시각 placeholder

- 상단: Coding Agent harness blocks
- 하단: Research Agent의 빈 칸과 물음표

발표 메모

- 이 슬라이드가 발표의 중심 주장이다.

11. DOE를 Harness 후보로 보기

- DOE는 무엇을, 어떤 순서로, 어떤 조합으로 실험할지에 규율을 준다.
- screening은 중요한 factor를 초기에 좁히는 데 유용하다.
- factorial thinking은 상호작용을 우연이 아니라 구조적으로 보게 만든다.
- sequential refinement와 robustness check는 후반 탐색을 더 disciplined하게 만든다.
- 따라서 DOE는 통계 기법이면서 동시에 agent harness 제안이 될 수 있다.

시각 placeholder

- DOE toolbox
- screening / interaction check / refinement / robustness

발표 메모

- DOE를 "통계 수업 내용"이 아니라 "agent 운영 규율"로 소개한다.

12. Agent loop에 무엇이 달라지는가?

- Vanilla Agent loop:
- idea → code change → run → score → repeat
- DOE-guided Agent loop:
- factor 정의 → screening → interaction check → refinement → robustness check → promote
- 차이는 최고 점수 하나보다도 experiment ordering과 해석 구조에 있다.

시각 placeholder

- before / after 비교 도식

발표 메모

- 이후 실험 비교가 뜬금없지 않게 보이도록 loop 차이를 먼저 명시한다.

13. 실험 설정

- Variant A: Vanilla Agent
- Variant B: DOE Agent
- Variant C: DOE + X Agent
- 여기서 `X` 는 이후 고정할 추가 전략 모듈이다.
- 평가는 best score만이 아니라 cost, robustness, portfolio behavior까지 함께 본다.

시각 placeholder

- 표: Variant / planning style / extra module / notes

발표 메모

- 이 비교는 prompt wording 대결이 아니라 harness 설계 비교라는 점을 분명히 한다.

14. 결과 요약

- Placeholder only: 실험 완료 후 채운다.
- 권장 지표:
- best score
- average improvement
- run count
- cost
- robustness pass rate

시각 placeholder

- 표:
- Variant | Best score | Avg improvement | Runs | Cost | Robustness pass
- Vanilla | TBD | TBD | TBD | TBD | TBD
- DOE | TBD | TBD | TBD | TBD | TBD
- DOE + X | TBD | TBD | TBD | TBD | TBD

15. 성능 향상 추이

- Placeholder only: 실험 완료 후 채운다.
- experiment budget 또는 run count 대비 best-so-far metric을 그린다.
- 수렴 속도, plateau 형태, sample efficiency를 비교한다.
- variant마다 한 곡선을 둔다.

시각 placeholder

- x축: budget 또는 run count
- y축: best-so-far metric
- 곡선: Vanilla / DOE / DOE + X

발표 메모

- 최종 점수보다 learning dynamics가 어떻게 바뀌는지를 보여주는 슬라이드다.

16. 실험 포트폴리오 또는 실패 구조

- Placeholder only: 실험 완료 후 채운다.
- Option A: `Tic / Tac / To` 비율
- Option B: invalid / no gain / unstable / robustness fail / promoted
- Option C: information gain vs. performance gain

시각 placeholder

- bar chart 또는 stacked-area chart

발표 메모

- DOE의 가치는 최고점뿐 아니라 탐색의 질을 바꾸는 데 있다는 점을 보여준다.

17. 왜 DOE-guided Agent는 다를 수 있는가?

- search space가 더 구조화된다.
- 중복되거나 정보량이 낮은 실험이 줄어든다.
- 상호작용을 우연히가 아니라 의도적으로 본다.
- opportunistic improvement와 robustness 확인을 분리할 수 있다.
- experiment history가 재사용 가능한 지식으로 축적된다.

시각 placeholder

- DOE discipline → exploration quality로 이어지는 5단계 인과 도식

발표 메모

- 결과 해석은 숫자 나열이 아니라 agent 행동 변화에 대한 설명이어야 한다.

18. 한계

- DOE가 literature review나 hypothesis generation 자체를 대체하진 못한다.
- open-ended research space는 깔끔한 factorization이 어렵다.
- large-scale training에서는 replication 비용이 크다.
- classical DOE는 discrete structural edit 공간에 그대로 맞지 않을 수 있다.
- evaluation harness가 약하면 DOE도 쉽게 흔들린다.

시각 placeholder

- factorization / cost / discrete structure / evaluation fragility 아이콘 목록

발표 메모

- DOE는 강한 후보이지 만능 해법은 아니라는 점을 분명히 한다.

19. 결론

- AutoML에서 Autoresearch로 갈수록 search space는 넓어진다.
- search space가 넓어질수록 더 강한 harness가 필요하다.
- DOE는 Research Agent의 experimentation harness로 매우 유력한 후보다.

> **다음 단계는 더 좋은 agent 자체보다 더 좋은 harness이다.**

시각 placeholder

- 3단계 ladder 또는 closing statement

발표 메모

- 결론은 agent 성능 승부보다 harness 설계의 중요성으로 닫는다.

20. References

- `awesome-autoresearch`
- `Awesome-Auto-Research-Tools`
- AutoML / MLOps 관련 기본 문헌
- AI Scientist / MLE-bench / MLAgentBench / MLR-Bench
- DOE / bandit / active learning 관련 문헌

시각 placeholder

- 2단 bibliography layout

발표 메모

- 공개 생태계, benchmark, 통계적 실험 설계를 함께 묶어 보여 준다.